CollegeChat

A Private Real-Time Chat Platform for Your College

Full Project Roadmap — Complete Build Guide

---

1. Project Overview

CollegeChat is a closed, real-time web chat application exclusively for students of your college. No outsider can register — access is controlled via a Secret College Code + Roll Number Pattern validation. Students can chat in rooms organized by branch and year, share confessions anonymously, discover lost items, and stay updated on events — all in one place.

[Goal] Build a production-grade chat app that becomes the default internal communication platform for your college.

[Key Constraint] Zero dependency on college email IDs or manual approval. Access is controlled by a secret code + roll number pattern only.

---

2. Tech Stack & Installations

Frontend

Technology	Purpose	Install Command
React (Vite)	UI Framework	npm create vite@latest frontend --template react
React Router v6	Client-side routing	npm install react-router-dom
Socket.io-client	Real-time chat (client)	npm install socket.io-client
Axios	HTTP API calls	npm install axios
React Context API	Global state (auth, user)	Built into React — no install
Tailwind CSS	Styling	npm install -D tailwindcss postcss autoprefixer
React Hot Toast	Notifications/toasts	npm install react-hot-toast

Technology	Purpose	Install Command
Lucide React	Icon library	npm install lucide-react
date-fns	Date/time formatting	npm install date-fns
emoji-picker-react	Emoji reactions picker	npm install emoji-picker-react

Backend

Technology	Purpose	Install Command
Node.js + Express	Server framework	npm install express
Socket.io	Real-time WebSockets	npm install socket.io
Mongoose	MongoDB ODM	npm install mongoose
JWT (jsonwebtoken)	Auth tokens	npm install jsonwebtoken
bcryptjs	Password hashing	npm install bcryptjs
cookie-parser	Read cookies on server	npm install cookie-parser
cors	Cross-origin requests	npm install cors
dotenv	Environment variables	npm install dotenv
express-rate-limit	Rate limiting / spam control	npm install express-rate-limit
nodemon	Auto-restart in dev	npm install -D nodemon
multer	File/image upload handling	npm install multer
cloudinary	Cloud image storage (avatars)	npm install cloudinary

Database & Cloud

Service	Purpose	How to Access
MongoDB Atlas	Cloud database (free tier)	mongodb.com/atlas
Cloudinary	Store profile pics / images	cloudinary.com (free tier)
Render	Deploy backend + frontend	render.com (free tier)

---

3. Folder Structure

Root

collegechat/

- backend/ ← Express + Socket.io server

- frontend/ ← React (Vite) app
- .gitignore
- README.md

Backend Folder Structure

backend/

- server.js ← Entry point, Express + Socket.io init
- config/
- db.js ← MongoDB connection
- cloudinary.js ← Cloudinary config
- models/
- User.js ← User schema
- Message.js ← Message schema
- Room.js ← Chat room schema
- Confession.js ← Anonymous confession schema
- LostFound.js ← Lost & found item schema
- Event.js ← College events schema
- routes/
- auth.routes.js ← Register, Login, Logout
- user.routes.js ← Profile, update, avatar
- room.routes.js ← Create/get rooms
- message.routes.js ← Fetch paginated messages
- confession.routes.js ← Post/get confessions
- lostfound.routes.js ← Post/get lost & found
- event.routes.js ← Post/get events
- admin.routes.js ← Admin controls
- controllers/
- auth.controller.js
- user.controller.js
- room.controller.js
- message.controller.js
- middleware/
- auth.middleware.js ← Verify JWT on protected routes
- admin.middleware.js ← Check if user is admin
- rateLimiter.js ← Rate limiting middleware
- socket/
- index.js ← All Socket.io events
- .env

Frontend Folder Structure

frontend/src/

- main.jsx ← App entry point
- App.jsx ← Routes setup
- components/
- auth/ ← Login.jsx, Register.jsx
- chat/ ← ChatWindow.jsx, MessageBubble.jsx, MessageInput.jsx
- rooms/ ← RoomList.jsx, RoomCard.jsx
- sidebar/ ← Sidebar.jsx, OnlineUsers.jsx
- profile/ ← ProfileCard.jsx, EditProfile.jsx
- confessions/ ← ConfessionCard.jsx, PostConfession.jsx
- lostfound/ ← LostFoundCard.jsx, PostItem.jsx
- events/ ← EventCard.jsx, PostEvent.jsx
- admin/ ← AdminPanel.jsx, UserTable.jsx
- shared/ ← Navbar.jsx, Loader.jsx, Modal.jsx, Avatar.jsx
- context/
- AuthContext.jsx ← Auth state globally
- SocketContext.jsx ← Socket.io instance globally
- hooks/
- useChat.js ← Chat logic hook
- useSocket.js ← Socket event listeners
- pages/
- LoginPage.jsx
- RegisterPage.jsx
- ChatPage.jsx
- ProfilePage.jsx
- AdminPage.jsx
- LostFoundPage.jsx
- ConfessionsPage.jsx
- EventsPage.jsx
- services/
- api.js ← Axios base instance
- auth.service.js
- message.service.js
- room.service.js
- utils/
- helpers.js ← Date formatters, validators
- constants.js ← Room names, roll regex pattern

---

4. Access Control System

How It Works — Two Layers

[Layer 1 — Secret College Code] Store the code in backend .env. On register, user must enter this code. Wrong code = rejected instantly. You can change it anytime by updating .env.

[Layer 2 — Roll Number Pattern Validation] Define a regex pattern matching your college's roll number format (e.g., 23CSE045). Backend validates using regex on register. No match = rejected.

Environment Variables (.env)

- COLLEGE_SECRET_CODE=MIT2025CHAT
- ROLL_NUMBER_REGEX=^(22|23|24|25)(CSE|IT|MECH|CIVIL|EE)\d{3}$
- JWT_SECRET=your_jwt_secret_key
- MONGODB_URI=your_mongodb_atlas_uri
- CLOUDINARY_CLOUD_NAME=your_cloudinary_name
- CLOUDINARY_API_KEY=your_key
- CLOUDINARY_API_SECRET=your_secret
- CLIENT_URL=https://your-frontend.onrender.com
- PORT=5000

[Security Note] Never commit .env to GitHub. Add it to .gitignore immediately. Set these as environment variables on Render during deployment.

---

5. Database Schema Design

Model	Key Fields	Notes
User	name, rollNumber, password, branch, year, avatar, isAdmin, isBanned, createdAt	Roll number is unique identifier
Message	text, sender (ref User), room (ref Room), reactions[], isEdited, isDeleted, createdAt	Soft delete — don't actually remove
Room	name, slug, description, type (public/branch/year), createdBy, isReadOnly	isReadOnly for announcements
Confession	text, isAnonymous (always true), reactions[], createdAt	No sender reference stored
LostFound	title, description, type (lost/found), contact, postedBy (ref User), isResolved	Contact is optional
Event	title, description, date, location, postedBy, isApproved	Admin approves before visible

---

6. Features — Core, Additional & Optional

Core Features (Must Build — MVP)

These are non-negotiable for the app to function:

- **Register with Secret Code + Roll Number Validation**
  - Reject registration if code is wrong or roll number format doesn't match
- **JWT Authentication with HTTP-only Cookies**
  - Secure login/logout, protected routes, auto-login on page refresh
- **Multiple Chat Rooms by Branch/Year**
  - Default rooms: General, CSE, IT, MECH, 2nd Year, 3rd Year, 4th Year
  - Each room is isolated — messages don't bleed across rooms
- **Real-Time Messaging via Socket.io**
  - Messages appear instantly for all connected users in the room
  - No page refresh needed
- **Message Persistence in MongoDB**
  - All messages stored — users who join later can see old chats
- **Message Pagination**
  - Load last 50 messages on open, scroll up to load older ones
- **Online Users List**
  - Show who's currently online in the sidebar
- **Typing Indicator**
  - Show 'XYZ is typing...' in real time
- **Basic User Profile**
  - Name, branch, year, avatar (upload to Cloudinary)

Additional Features (Build in Phase 2-3)

These elevate the app from MVP to something students actually love:

- **Emoji Reactions on Messages**
  - Click a message → react with emoji → updates in real time for everyone
- **Message Edit & Delete**
  - Soft delete — message shows 'This message was deleted'
- **@Mentions + Notifications**
  - Tag @username in a message — they get a notification badge
- **Announcements Room (Read-Only)**
  - Only admins can post — perfect for official college notices
- **Confessions Room**
  - 100% anonymous — no sender stored in database at all
  - React to confessions with emojis
- **Lost & Found Board**
  - Post lost/found items with optional contact info
  - Mark item as resolved when found

- **Event Board**
  - Students post upcoming fests, hackathons, internship deadlines
  - Admin approval before posts go live
- **Read Receipts / Last Seen**
  - 'Seen by 5 people' or 'Last seen 2 hours ago'
- **Rate Limiting**
  - Max 10 messages per 10 seconds per user to prevent spam
- **Admin Panel**
  - Ban/unban users, pin messages, view usage stats, manage rooms

Optional / Advanced Features (Phase 4-5)

Build these to make it viral and technically impressive:

- **Anonymous Compliments System**
  - Send a compliment to a specific roll number anonymously
  - They get notified: 'Someone sent you a compliment!'
- **GIF Support via Giphy API**
  - Search and send GIFs inside chat
- **Image Sharing in Chat**
  - Upload image in chat → stored on Cloudinary → displayed inline
- **Message Search**
  - Search across all messages in a room by keyword
- **Student Directory**
  - Browse all registered students — filter by branch/year
- **Dark Mode**
  - Toggle between light and dark theme, saved to localStorage
- **PWA (Progressive Web App)**
  - Make it installable on mobile home screen like a native app
  - Push notifications when mentioned or someone sends a message
- **Buy/Sell Marketplace**
  - Students list old books, cycles, laptops for sale
- **Secret Santa / Group Matching**
  - Fun feature for festivals — random secret gifting partner assignment

---

7. Things You May Have Missed

[Critical — CORS Configuration] Since frontend and backend are on different domains/ports, you must configure CORS properly with credentials: true and exact origin URL. Wrong CORS = cookies won't work.

[Critical — Socket.io Auth] Socket connections must also be authenticated, not just HTTP routes. Verify JWT on every socket connection using socket.handshake.auth.token in the connection middleware.

[Important — Secret Code Rotation] Plan a way to update the college secret code when it leaks. Changing the .env value and redeploying should be enough — but document this process.

[Important — Profanity Filter] Add a basic word filter for the chat. Students will test limits. Use the 'bad-words' npm package on the backend before saving messages to DB.

[Important — Mobile Responsiveness] Most students will use this on phones. Make Tailwind CSS layouts fully responsive from Day 1, not as an afterthought.

[Important — Socket Room Management] When a user switches chat rooms, you must call socket.leave(oldRoom) and socket.join(newRoom). Missing this = messages broadcasting to wrong rooms.

[Consider — Scroll Behavior] Auto-scroll to latest message when new message arrives, but don't auto-scroll if the user has scrolled up to read history — this is annoying.

[Consider — Message Timestamps] Show relative time ('2 min ago') for recent messages, absolute time for older ones. Use the date-fns library for this.

[Consider — Error Boundaries] Add React error boundaries so one broken component doesn't crash the entire app. Critical for production.

[Consider — Reconnection Logic] Socket.io auto-reconnects but you need to handle the UI — show a 'Reconnecting...' banner when connection drops instead of silently failing.

---

8. Phase-Wise Build Roadmap

Phase	What to Build	Duration (Estimate)	Status
Phase 1 — Foundation	Project setup, folder structure, MongoDB connection, Express server, Socket.io init, CORS, .env setup	2–3 days	Start Here
Phase 2 — Auth System	Register route with secret code + roll number validation, JWT login/logout, auth middleware, cookie setup	3–4 days	Core Security

Phase	What to Build	Duration (Estimate)	Status
Phase 3 — Chat Core	Room model + routes, Socket.io messaging, message persistence, typing indicator, online users list, basic React UI	5–7 days	Heart of App
Phase 4 — Polish MVP	Message pagination, emoji reactions, edit/delete messages, profile page + avatar upload, rate limiting, toast notifications	4–5 days	MVP Ready
Phase 5 — Features	Confessions room, Lost & Found, Event board, Announcements room, @Mentions, Admin panel	5–7 days	Phase 2 Features
Phase 6 — Advanced	Anonymous compliments, GIF support, image sharing in chat, student directory, message search, dark mode	5–7 days	Optional
Phase 7 — Deploy	Deploy backend to Render, frontend to Render/Vercel, MongoDB Atlas whitelisting, env vars on Render, test all flows	2–3 days	Production
Phase 8 — PWA	Service worker, manifest.json, push notifications, installable on mobile	3–4 days	Advanced Optional

---

9. How to Start — Day 1 Checklist

Step 1 — Create GitHub Repo

- Create a new repo called collegechat on GitHub
- Initialize with a README and .gitignore (Node template)
- Clone it locally

Step 2 — Setup Backend

- Inside repo, run: mkdir backend && cd backend && npm init -y
- Install all backend dependencies listed in Section 2
- Create the full folder structure from Section 3
- Create .env file with all variables from Section 4
- Add .env to .gitignore immediately
- Write server.js with Express + Socket.io + MongoDB connection

Step 3 — Setup Frontend

- From repo root: npm create vite@latest frontend -- --template react

- cd frontend && npm install
- Install all frontend dependencies from Section 2
- Setup Tailwind CSS (follow official Vite + Tailwind guide)
- Create the full folder structure from Section 3
- Setup Axios base instance in services/api.js with withCredentials: true

Step 4 — Start Coding Order

- 1. User model (models/User.js) — the foundation of everything
- 2. Register route with secret code + roll validation
- 3. Login route returning JWT as HTTP-only cookie
- 4. Auth middleware to protect routes
- 5. Room model and default rooms seed script
- 6. Socket.io connection with JWT verification
- 7. Send/receive message via Socket.io
- 8. Save message to MongoDB on receive
- 9. React chat UI — connect socket, show messages, send on Enter
- 10. Paginate messages on room open

[Pro Tip] Test every backend route in Postman or Thunder Client before touching the frontend. This saves hours of debugging.

---

10. Deployment Checklist

MongoDB Atlas

- Create free cluster on mongodb.com/atlas
- Whitelist 0.0.0.0/0 (allow all IPs — required for Render)
- Copy connection URI to backend .env

Cloudinary

- Create free account at cloudinary.com
- Copy Cloud Name, API Key, API Secret to backend .env

Render — Backend

- Create a new Web Service on Render
- Connect GitHub repo, set root directory to backend/
- Build command: npm install

- Start command: node server.js
- Add all .env variables in Render's Environment section
- Copy the deployed backend URL

Render — Frontend

- Create a new Static Site on Render
- Root directory: frontend/
- Build command: npm run build
- Publish directory: dist
- Add _redirects file in frontend/public/ with: /* /index.html 200
- Set VITE_API_URL env variable to your backend Render URL

[Cookie Fix for Production] Set cookie options on backend: secure: true, sameSite: 'none'. This is required when frontend and backend are on different domains.

---

11. Project Summary Card

Item	Detail
Project Name	CollegeChat
Type	Full-Stack Real-Time Web Application
Frontend	React (Vite) + Tailwind CSS + Socket.io-client
Backend	Node.js + Express + Socket.io
Database	MongoDB Atlas (Mongoose)
Auth	JWT + bcryptjs + HTTP-only Cookies
Real-Time	Socket.io WebSockets
Image Storage	Cloudinary
Deployment	Render (backend + frontend) + MongoDB Atlas
Access Control	Secret College Code + Roll Number Regex Validation
Total Estimated Time	4–6 weeks (part-time) / 2–3 weeks (full-time)
Difficulty Level	Intermediate-Advanced (you're ready for this)

Build it. Ship it. Make your college talk about it.

— CollegeChat Roadmap by Manas